

# PrePro **Auto**

## Complete Project & User Guide

v1.0.0b5 • 2026 • Author: Shivanshu Pandey

---

```
pip install prepro-auto
```

|               |              |                 |                   |
|---------------|--------------|-----------------|-------------------|
| <b>243</b>    | <b>5</b>     | <b>5</b>        | <b>18</b>         |
| Tests passing | AI Providers | Cleaning Stages | Transform Presets |

RAM-aware • Max Upload • Reproducible & Auditable

| Section 1 — Project Deep-Dive                                          | Section 2 — Complete User Guide                                                            |
|------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| Architecture, methods overview, dataset limits, and design philosophy. | Step-by-step: notebook interface, web UI, every function explained, all outputs described. |

## Contents

|           |                                             |           |
|-----------|---------------------------------------------|-----------|
| <b>1</b>  | <b>What is PrePro Auto?</b>                 | <b>2</b>  |
| <b>2</b>  | <b>Project Goal &amp; Design Philosophy</b> | <b>2</b>  |
| <b>3</b>  | <b>Target Users</b>                         | <b>3</b>  |
| <b>4</b>  | <b>System Architecture</b>                  | <b>3</b>  |
| <b>5</b>  | <b>End-to-End Workflow</b>                  | <b>4</b>  |
| <b>6</b>  | <b>How It Works (Methods at a Glance)</b>   | <b>13</b> |
| <b>7</b>  | <b>Dataset Size Limits</b>                  | <b>13</b> |
| <b>8</b>  | <b>Installation</b>                         | <b>15</b> |
| <b>9</b>  | <b>Data Privacy</b>                         | <b>15</b> |
| <b>10</b> | <b>Notebook Interface — All Functions</b>   | <b>16</b> |
| <b>11</b> | <b>Web UI — Step-by-Step Guide</b>          | <b>18</b> |
| <b>12</b> | <b>All Outputs Explained</b>                | <b>20</b> |
| <b>13</b> | <b>AI Providers Setup</b>                   | <b>20</b> |
| <b>14</b> | <b>Troubleshooting</b>                      | <b>21</b> |

---

## SECTION 1

### Project Deep-Dive

---

#### 1. What is PrePro Auto?

**PreProAuto** is an AI-assisted, human-in-the-loop tabular data preprocessing tool. It combines a statistical preprocessing engine, a local web-based workbench UI, a Jupyter notebook bridge, and optional AI assistance from five LLM providers — all behind a single `pip install`.

The core idea is simple: instead of writing dozens of pandas / sklearn lines to clean a dataset — guessing at imputation strategies, eyeballing outliers, picking scalers from memory — you load your data into the workbench, review AI-generated decision cards for each issue found, approve or override them, and walk away with a clean dataset plus a runnable Python pipeline that reproduces every step.

#### Key facts at a glance:

| Property          | Value                                                                                                                                                                   |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Package name      | <code>prepro-auto</code>                                                                                                                                                |
| Install           | <code>pip install prepro-auto</code>                                                                                                                                    |
| Version           | 1.0.0b5 (public beta)                                                                                                                                                   |
| License           | MIT — free for personal and commercial use                                                                                                                              |
| Author            | Shivanshu Pandey ( <a href="https://github.com/Chilliflex">github.com/Chilliflex</a> )                                                                                  |
| Python            | 3.10 / 3.11 / 3.12                                                                                                                                                      |
| Tests             | 243 passing (pytest, CI on every push)                                                                                                                                  |
| Dependencies      | FastAPI, pandas, scikit-learn, scipy, reportlab, uvicorn + more                                                                                                         |
| AI (optional)     | Groq, OpenAI, Anthropic, Gemini, Mistral — all optional                                                                                                                 |
| Engine runs local | Preprocessing engine: 100% local, no telemetry.<br>Optional AI sends column names, dtypes & a few sample values (PII-masked) to your chosen provider only when enabled. |

#### 2. Project Goal & Design Philosophy

The goal is to eliminate the most time-consuming and error-prone part of every machine learning project: cleaning the raw data. Data scientists spend a large proportion of project time on preprocessing, and most of that time is spent on decisions that are statistically derivable yet written from scratch each time.

Three design principles guide every decision in the codebase:

**Stats beats ML/DL when the answer is derivable.** The correct scaler for a Gaussian column is `StandardScaler` — no neural network needs to learn that. **PreProAuto** uses statistical tests (Shapiro-Wilk, KS, IQR, Pearson/Spearman, PSI) where the answer is mathematically derivable, and only involves ML models (Isolation Forest, KNN, MICE) where the data itself must inform the decision.

**The human is always in control.** Every recommendation is a decision card: the system proposes, the human approves, overrides, or skips. Nothing changes the data without explicit approval. Every change is a versioned snapshot with full undo/redo.

**Reproducibility is not optional.** Every session produces a runnable Python pipeline script that reproduces the exact cleaning with no **PreProAuto** dependency. The exported script is **transform-only**: all parameters (means, medians, scaler statistics, category maps, target means) are fitted once inside **PreProAuto** and baked into the script as literal values — the script never re-fits on the data it is given. See Section 5 for the full reproducibility guarantee.

### 3. Target Users

| User                        | How they use PrePro Auto                                                                                                                                                            |
|-----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Data Scientist (ML / DL)    | Launch from Jupyter: <code>prepro_auto.launch_file(path)</code> . Clean visually, then <code>session.current()</code> feeds straight into <code>model.fit()</code> . Zero file I/O. |
| Data Analyst                | Run the web UI ( <code>prepro_auto</code> in terminal), upload CSV, step through the guided 6-step flow, download clean CSV and audit PDF.                                          |
| ML Engineer / Data Engineer | Use the exported <code>pipeline.py</code> in Airflow / Prefect / GitHub Actions. Cleaning is automated and reproducible in production.                                              |
| Student / Researcher        | Free, offline, no API key required. A complete preprocessing environment for academic datasets with full explainability of every decision.                                          |
| Compliance / Audit role     | The audit PDF lists every transformation with parameters, before/after stats, and who approved it — a complete provenance trail.                                                    |

### 4. System Architecture

**PreProAuto** has a layered architecture. All layers run locally in a single process when installed via pip — no Docker, no cloud, no network required.

| Layer                           | Technology                      | Responsibility                                                       |
|---------------------------------|---------------------------------|----------------------------------------------------------------------|
| Notebook bridge                 | prepro_auto/__init__.py         | quickclean(), launch(), launch_file(), Session(), set_api_key()      |
| REST API                        | FastAPI + uvicorn               | All /api/v1/ endpoints; consumed by both UI and notebook             |
| Preprocessing engine            | app/preprocessing/              | 5 cleaning stages, profiler, transforms, drift                       |
| Service layer                   | app/services/                   | Orchestrates stages, manages decision cards, export                  |
| Privacy layer ( <i>new b5</i> ) | app/services/privacy.py         | PII detection & masking before any value reaches the AI layer        |
| Versioning                      | SQLite + app/db/                | Every snapshot is a committed version; undo/redo                     |
| Storage                         | Local filesystem                | Raw uploads and snapshots stored in ./storage/                       |
| Web UI                          | app/web/*.html + FastAPI static | 6-step guided workbench served at /workbench                         |
| AI client                       | app/services/llm_client.py      | Provider-agnostic LLM wrapper; optional                              |
| Export engine                   | app/services/export_service.py  | CSV/Parquet, audit PDF, <b>transform-only</b> pipeline.py generation |

### Data flow diagram:

```

User Input (file / DataFrame)
|
[prepro_auto bridge / Web Upload]    (or quickclean() for a headless one-call run)
|
FastAPI REST layer (/api/v1/)
|
Preprocessing Engine
  Profiler (type inference, quality score)
  Missing Values (MCAR/MAR/MNAR + KNN/MICE/median/mode) --> fitted fill values stored
  Outlier Detection (IQR + Hampel + Isolation Forest)
  Scaling (Shapiro-Wilk/KS -> StandardScaler/RobustScaler/Box-Cox...) --> fitted params
    stored
  Correlation + Leakage (Pearson/Spearman, ID detection)
  Encoding (cardinality-routed: label/ordinal/OHE/frequency/target) --> fitted maps stored
|
Decision Cards (human reviews: Approve / Override / Skip)
|
Execute Stage -> Fitted state persisted into TransformationLog.params._fitted
                  -> New Snapshot (versioned, undoable)
|
[Optional AI layer]
  Privacy filter: PII-masked column names + dtypes + small sample (never full dataset)
  --> LLM provider (Groq / OpenAI / Anthropic / Gemini / Mistral)
|
Outputs: cleaned DataFrame | CSV/Parquet | Audit PDF | pipeline.py (transform-only) | Drift
report

```

## 5. End-to-End Workflow

## Step 1 — Upload / Load

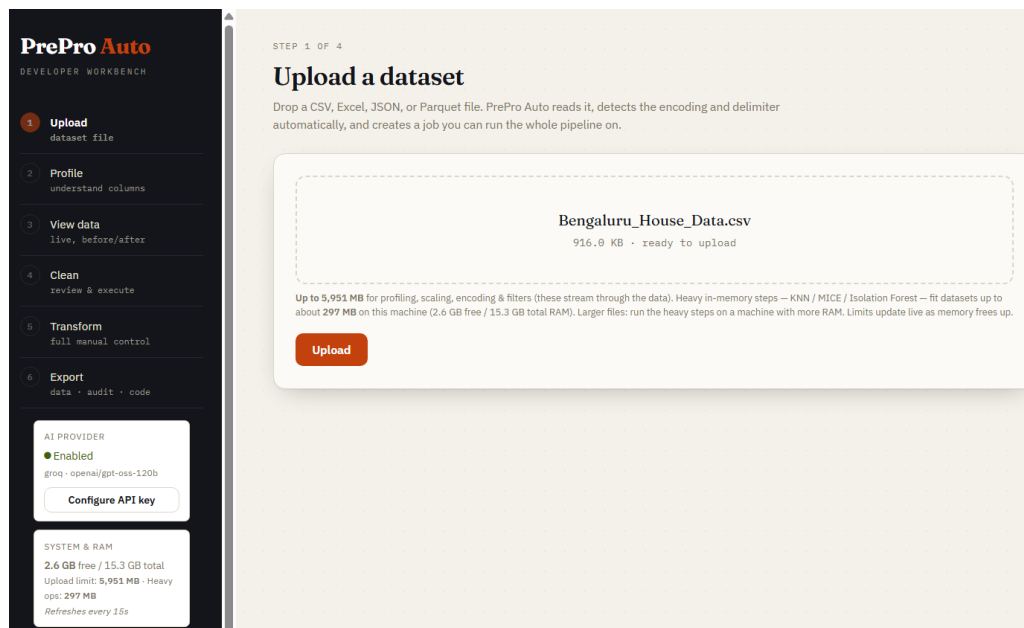


Figure 1: Upload / Load interface.

Drop a CSV, Parquet, Excel, or JSON file onto the upload zone (or call `prepro_auto.launch_file(path)` in a notebook). Encoding and delimiter are automatically detected. The sidebar RAM panel displays live memory availability and upload limits for the current machine.

*Example:* The Bengaluru\_House\_Data.csv dataset (916 KB) has been successfully loaded. The sidebar shows 2.6 GB free out of 15.3 GB total RAM, along with recommended limits for streaming and memory-intensive operations.

## Step 2 — Profile

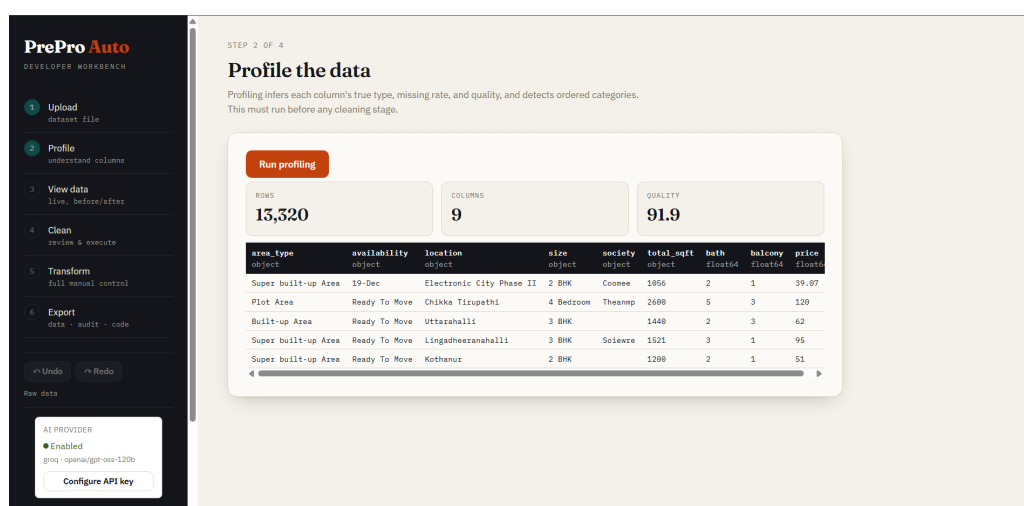


Figure 2: Profile interface.

The profiler performs automatic dataset analysis by computing semantic data types, missing-value percentages, cardinality statistics, representative sample values, and an overall data-quality

score. It **runs automatically** the moment you open the Profile step (no button). This stage is observational only and does not modify the dataset.

*Example:* Per-column semantic types, missing rates, cardinality statistics, and dataset-level quality metrics are computed in a single profiling pass.

### Step 3 — View Dataset

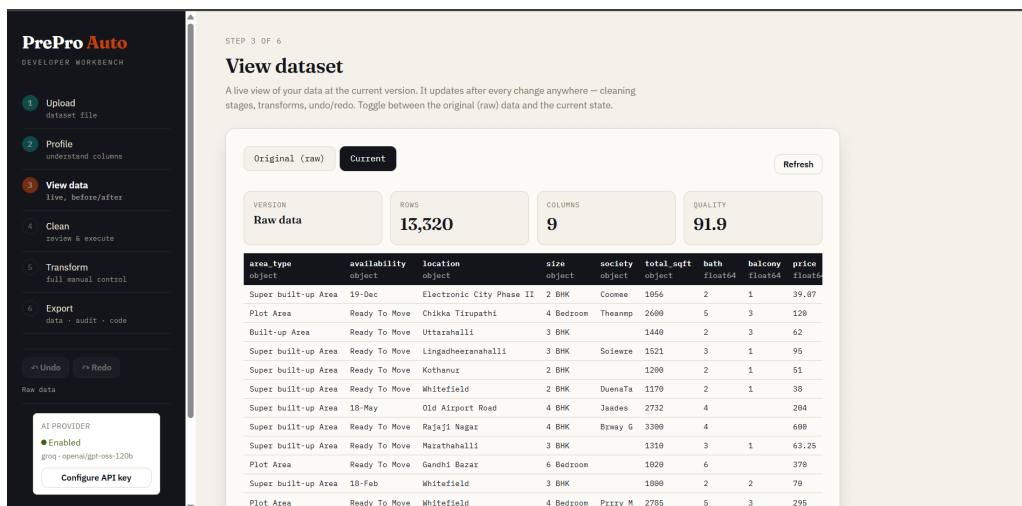


Figure 3: View Dataset interface.

The dataset viewer provides a live tabular representation of the current data state. Users can switch between the original uploaded dataset and the cleaned version at any time. Dataset metadata such as row count, column count, and quality score are updated automatically after every transformation.

*Example:* The displayed dataset contains 13,320 rows and 9 columns with a quality score of 91.9.

### Step 4 — Clean

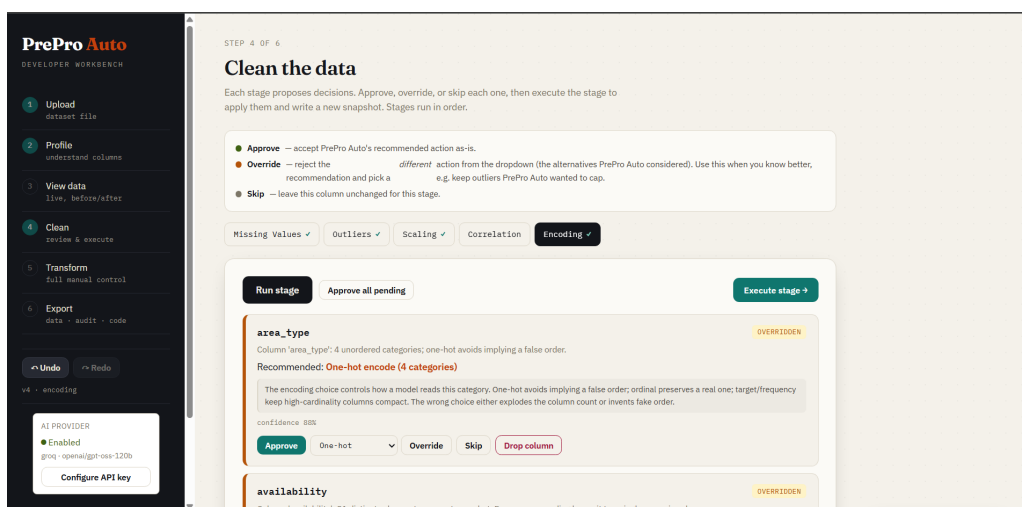


Figure 4: Clean interface.

The cleaning stage operates through five sequential sub-stages: Missing Values, Outliers, Scaling, Correlation, and Encoding. For each column, the system generates a decision card containing the recommended action, a confidence percentage, a plain-English explanation of the business impact, and a list of alternative strategies. The user reviews each card and selects one of four responses: Approve, Override, Skip, or Drop column. **Each stage runs automatically when you select its tab** — there is no separate “Run stage” button. Executing a stage commits all approved decisions as a new versioned, undoable snapshot *and automatically advances to and runs the next stage*.

**b5 — Fitted state is now persisted at this step.** When you execute a stage, the parameters **PreProAuto** computed — e.g. the median used to fill missing values, the mean and standard deviation the scaler fitted, the category-to-integer map an encoder built, the smoothed (M-estimate) per-category target means — are stored in `TransformationLog.params._fitted`. The exported `pipeline.py` reads these stored values to reproduce the transform without ever re-fitting on new data.

*Example:* The Encoding stage is active. The `area_type` column has been flagged with an **OVER-RIDDEN** badge, indicating the user rejected the system’s One-hot encoding recommendation (88% confidence).

## Step 5a — Transform: Preset Operations

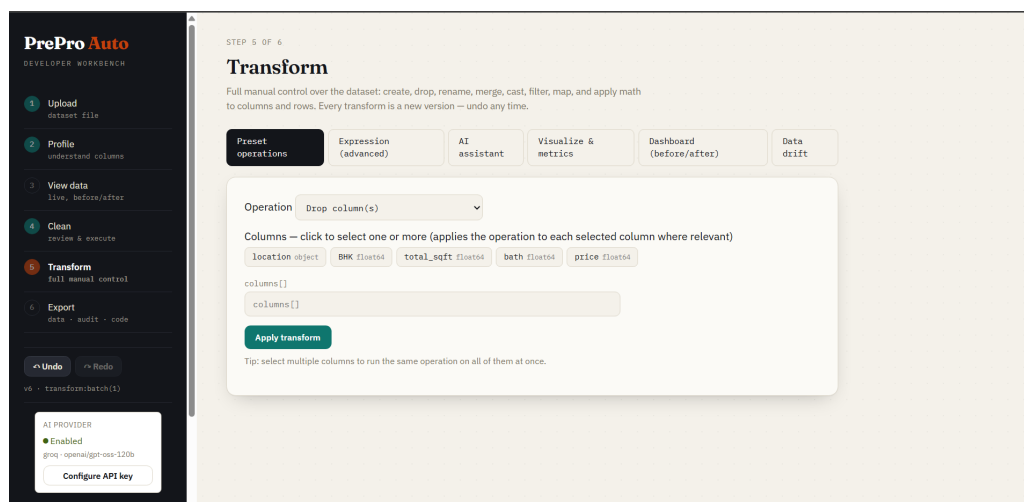


Figure 5: Transform: Preset Operations interface.

The Preset Operations tab provides 18 built-in, point-and-click data transformations that require no coding. Available operations include: rename, drop, cast (type conversion), fill missing values, filter rows, merge columns, arithmetic operations (+, -, \*, /), map values, string transformations (uppercase, lowercase, strip, replace), regex extraction, **group-aggregate (group-by feature engineering)**, sort, deduplicate, and extract number from text. Multiple columns can be selected simultaneously to apply the same operation in a single undoable step.

The **group-aggregate** preset computes, for each row, the aggregate (mean / sum / count / min / max / median / std / nunique) of a value column over its group and broadcasts it back as a new feature column — e.g. *average price per location*.



## Step 5b — Transform: Expression (Advanced)

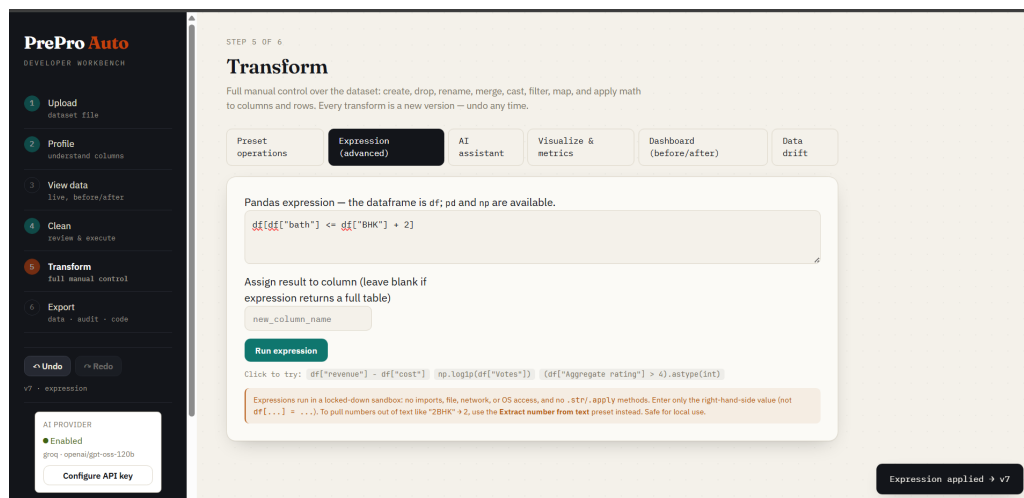


Figure 6: Transform: Expression (Advanced) interface.

The Expression Editor allows users to write pandas expressions against the current DataFrame. The dataframe is exposed as `df`; both `pd` (pandas) and `np` (NumPy) are available. Expressions are validated against a restricted namespace and a denylist of dangerous tokens (imports, file/OS/network access, `.apply`, `.eval`, loops, lambdas, etc.) before execution. Every successful expression application is saved as a new undoable version. See [SECURITY.md](#) for the full mechanism and trust-boundary description.

**Trust boundary:** The expression sandbox is designed for a **single-user, local-machine** context — the person running the server is the same person whose expressions are evaluated. It is **not** hardened for untrusted multi-tenant input. For multi-user or hosted deployments, run each instance in an isolated container.

## Step 5c — Transform: AI Assistant

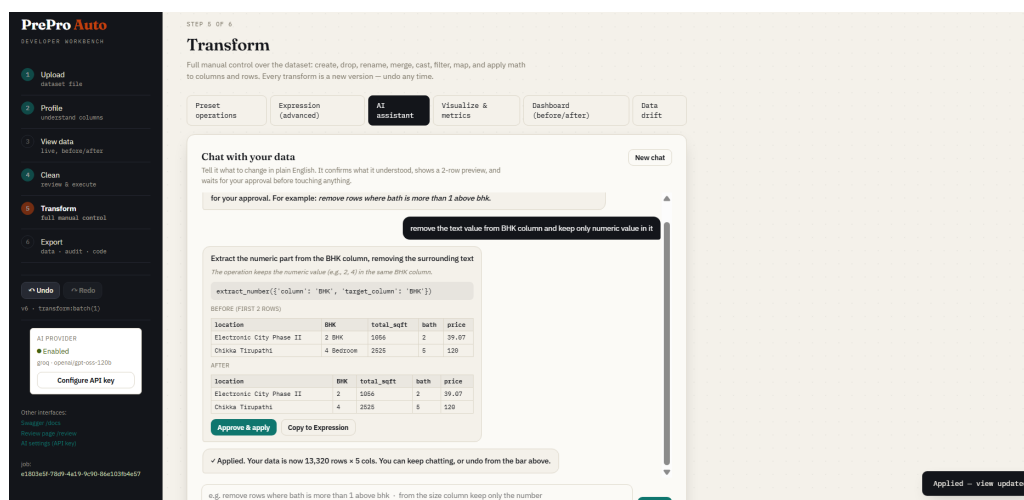


Figure 7: Transform: AI Assistant interface.

The AI Assistant tab enables natural-language data transformation. The user describes the desired change in plain English; the assistant interprets the request, generates the corresponding pandas operation, displays the inferred operation code, and presents a two-row before/after preview. No changes are applied until the user explicitly clicks **Approve & apply**.

**b5 — Privacy:** Before sending anything to the LLM provider, **PreProAuto** passes all column names and sample values through `app/services/privacy.py`. Emails, phone numbers, long ID numbers, and card-like patterns are masked to «email»/«phone»/«id»/«card». Columns whose names suggest PII (*email*, *ssn*, *name*, *address*, *phone*, *dob*, etc.) have all their sample values replaced with «redacted». The full dataset is **never** uploaded; only column names, dtypes, and a few masked sample values are sent. Control this with two settings in `app/core/config.py`:

LLM\_MASK\_PII                      Default:  
on

Redact PII-looking  
values before egress

LLM\_SEND\_VALUES                  Default:  
on

Set False to send **no**  
raw values at all

## Step 5d — Transform: Visualize & Metrics

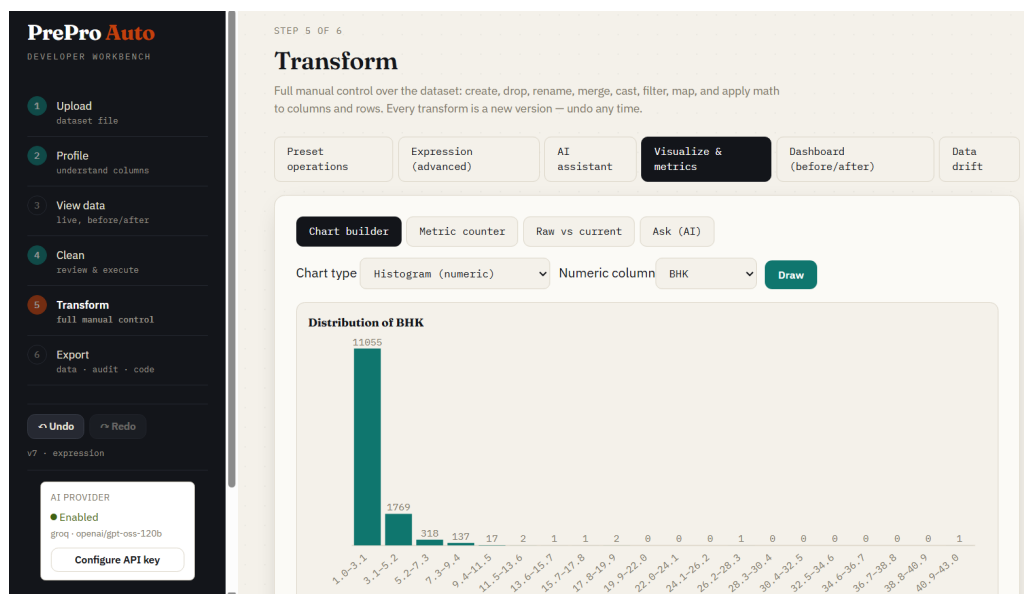


Figure 8: Transform: Visualize & Metrics interface.

The Visualize & Metrics tab provides live, in-workbench charting against the current dataset version. The Chart Builder supports histograms (numeric), bar charts (categorical), and scatter plots. The Metric Counter evaluates user-defined conditions. The Raw vs Current view overlays distributions before and after cleaning. The Ask (AI) option generates a chart from a natural-language description.

## Step 5e — Transform: Dashboard (Before and After)

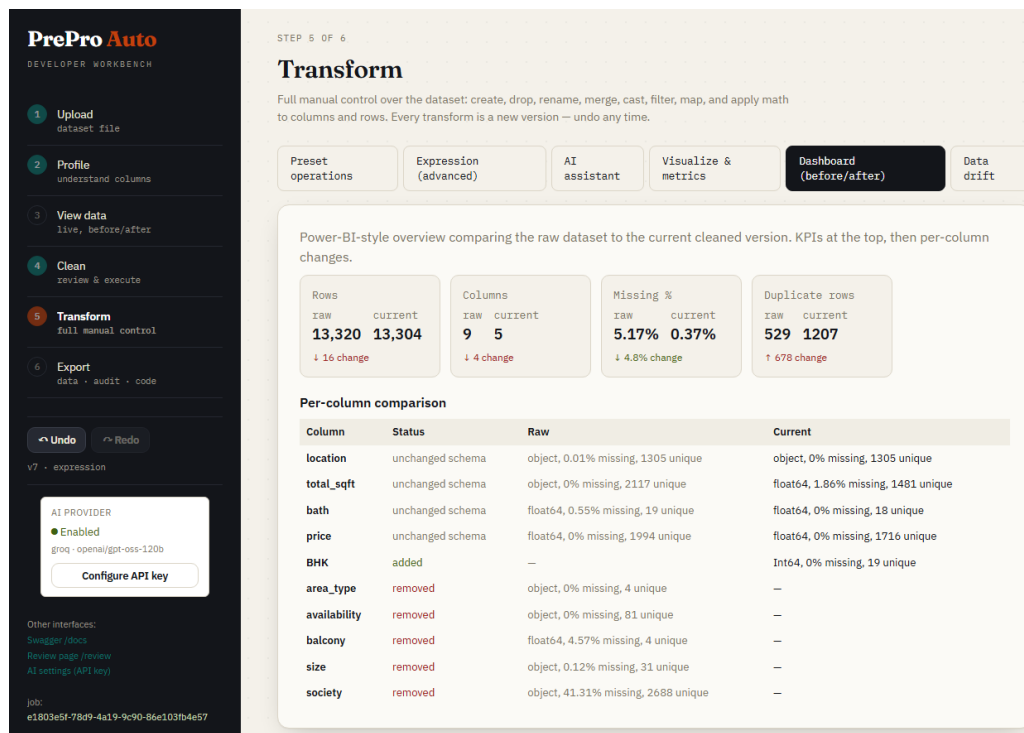


Figure 9: Transform: Dashboard (Before and After) interface.

The Dashboard tab delivers a Power-BI-style summary comparing the raw uploaded dataset to the current cleaned version. Four KPI tiles at the top display row count, column count, missing-value percentage, and duplicate row count, each showing the raw value, the current value, and the net change.

## Step 5f — Transform: Data Drift

**PrePro Auto**  
DEVELOPER WORKBENCH

STEP 5 OF 6

### Transform

Full manual control over the dataset: create, drop, rename, merge, cast, filter, map, and apply math to columns and rows. Every transform is a new version — undo any time.

Preset operations | Expression (advanced) | AI assistant | Visualize & metrics | Dashboard (before/after) | **Data drift**

**Compare two datasets for distribution drift**  
Upload your **reference** dataset (e.g. last month's training data) and a **new** dataset (e.g. recent production data). PrePro Auto computes per-column PSI and KS test for numerics, PSI for categoricals. No model required — works on any two CSV / Parquet / Excel / JSON files.

① **Reference dataset**  
Baseline / training / last period  
Choose File | Bengaluru\_House\_Data.csv  
Picked: Bengaluru\_House\_Data.csv

② **New dataset**  
Recent / production / current period  
Choose File | bhp.csv  
Picked: bhp.csv

**Detect drift**

**SIGNIFICANT DRIFT**  
1 significant • 0 moderate • 4 stable • 5 compared (13,320 ref / 13,200 new)

Schema differs — only in reference: area\_type, availability, balcony, society • only in new: bhk, price\_per\_sqft

| Column     | Type        | PSI     | Severity    | Detail                                        |
|------------|-------------|---------|-------------|-----------------------------------------------|
| total_sqft | categorical | 33.6176 | significant | 20 new categories (99.3% of new rows)         |
| location   | categorical | 0.008   | stable      | no new categories                             |
| size       | categorical | 0.0001  | stable      | no new categories                             |
| bath       | numeric     | 0       | stable      | ref mean 2.6926 → new mean 2.6911, KS p=1     |
| price      | numeric     | 0       | stable      | ref mean 112.5656 → new mean 112.2762, KS p=1 |

PSI < 0.10 stable • 0.10–0.25 moderate • ≥ 0.25 significant (industry-standard threshold). Drift means a model trained on the reference data may behave differently on the new data — investigate or retrain.

**AI PROVIDER**  
● Enabled  
groq · openai/gpt-oss-120b  
Configure API key

**SYSTEM & RAM**  
2.4 GB free / 15.3 GB total  
Upload limit: 5,318 MB · Heavy ops: 265 MB  
Refreshes every 15s

Other interfaces:  
Debugger (docs)  
Review page (review)  
AI settings (API key)

Figure 10: Transform: Data Drift Detection interface.

The Data Drift tab allows users to upload a second dataset and compare its column-level distributions against the reference dataset. The engine computes the Population Stability Index (PSI) for both numeric and categorical columns and the Kolmogorov-Smirnov (KS) two-sample statistic for numeric columns. Results are classified into three severity bands: stable ( $\text{PSI} < 0.10$ ), moderate ( $\text{PSI} 0.10\text{--}0.25$ ), and significant ( $\text{PSI} \geq 0.25$ ).

## Step 6 — Results & Export

**PrePro Auto**  
DEVELOPER WORKSPACE

- 1 Upload  
dataset file
- 2 Profile  
understand columns
- 3 View data  
1 row, before/after
- 4 Clean  
remove & rename
- 5 Transform  
full manual control
- 6 Export  
data • audit • code

Links: [Links](#) [Docs](#)

API PROVIDER  
☒ Enabled  
 gcp-openai@gcp-oss-120b  
[Configure API key](#)

Other interfaces:  
[Swagger \(link\)](#)  
[Review page \(review\)](#)  
[API settings \(API key\)](#)

job:  
 e1832e27-7b29-4a70-8c93-66a123b4a07

### Results & export

See exactly what preprocessing changed — the quality score, the column types before vs after, and a data sample — then download your deliverables.

**Before & after**  
 Comparison of the raw upload against the cleaned result. [Refresh](#)

**91.9**  
 BEFORE - RAW  
 12,328 rows • 9 cols

→  
 ▲ +6

**97.9**  
 AFTER - CLEANED - VT  
 12,386 rows • 5 cols

**Column types: before & after**

| COLUMN       | BEFORE TYPE         | AFTER TYPE           |
|--------------|---------------------|----------------------|
| area_type    | categorical_nominal | + (dropped)          |
| availability | categorical_nominal | + (dropped)          |
| location     | unknown             | + unknown            |
| size         | categorical_nominal | + (dropped)          |
| society      | unknown             | + (dropped)          |
| total_sqft   | unknown             | + numeric_continuous |
| bath         | numeric_discrete    | + numeric_discrete   |
| balcony      | numeric_discrete    | + (dropped)          |
| price        | numeric_continuous  | + numeric_continuous |
| BHK          | (new)               | + numeric_discrete   |

**Data sample: before**

| area_type           | availability  | location                 | size      | society | total_sqft | bath | balcony | price |
|---------------------|---------------|--------------------------|-----------|---------|------------|------|---------|-------|
| Super built-up Area | 19-Dec        | Electronic City Phase II | 2 BHK     | Cosmos  | 1056       | 2    | 1       | 29.87 |
| Plot Area           | Ready To Move | Chikka Tirupathi         | 4 Bedroom | Theanup | 2600       | 5    | 3       | 120   |
| Built-up Area       | Ready To Move | Uttarahalli              | 3 BHK     |         | 1660       | 2    | 3       | 62    |
| Super built-up Area | Ready To Move | Lingadheeranahalli       | 3 BHK     | Selowze | 1521       | 3    | 1       | 95    |
| Super built-up Area | Ready To Move | Kothanur                 | 2 BHK     |         | 1200       | 2    | 1       | 51    |

**Data sample: after**

| location                 | BHK | total_sqft | bath | price |
|--------------------------|-----|------------|------|-------|
| Electronic City Phase II | 2   | 1056       | 2    | 29.87 |
| Chikka Tirupathi         | 4   | 2600       | 5    | 120   |
| Uttarahalli              | 3   | 1660       | 2    | 62    |
| Lingadheeranahalli       | 3   | 1521       | 3    | 95    |
| Kothanur                 | 2   | 1200       | 2    | 51    |

**Download**  
 Take your results away.

**Clean data**  
 The processed dataset as CSV or Parquet.
 

CSV ▼
 [Download](#)

**Audit report**  
 A PDF listing every transformation and who decided it.
 [Download PDF](#)

**Pipeline code**  
 A standalone Python script reproducing the cleaning.
 [Download .py](#)

Figure 11: Results & Export interface.

The Results & Export step provides a final summary of all preprocessing activity and three downloadable artefacts: the cleaned dataset (CSV or Parquet), a full Audit PDF, and a standalone pipeline.py script.

### Reproducibility guarantee (b5) — no train/serve skew, no leakage.

The exported pipeline.py is **transform-only**. Every parameter each step needs was **fitted once inside PreProAuto** and is baked into the script as a literal value. The script never re-fits on the data it is given. Consequences:

- Running on a held-out / test set applies the *training* statistics — no test-set leakage, no train/serve skew.
- One-hot encoding always emits the **same columns in the same order**, regardless of which categories appear in new data (missing → 0, unseen → `__other__`).
- Target encoding uses M-estimate **smoothing** — rare-category means are shrunk toward the global mean, curbing overfitting and target leakage — and reuses the stored map (no recompute, no train/serve skew on new data).
- KNN/MICE: the pipeline reuses the **median learned at clean time**. (Full KNN/MICE cannot be deterministically re-applied to arbitrary new rows without skew; pickle-sidecar export is on the roadmap.)
- Input format is auto-detected: CSV, Parquet, Excel (.xlsx/.xls), and JSON are all handled — not just CSV.

Verified by `tests/test_reproducibility.py` and an end-to-end run on held-out data. Run: `python pipeline.py raw.csv ready.csv`

*Example:* The dataset quality score improved from 91.9 (raw, 13,320 rows, 9 columns) to 97.9 (cleaned, 13,304 rows, 5 columns), a gain of +6 points.

## 6. How It Works (Methods at a Glance)

**PreProAuto** uses only field-standard, transparent methods — no black boxes. Where the right answer is derivable from a column's distribution it uses a statistical test; machine learning is used only where the data itself must inform the choice. No deep learning is used.

| Stage                 | Methods used                                                                                                                                                                              |
|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Profiling & typing    | Regex / match-rate type inference; dictionary + heuristic semantics (optional AI hint).                                                                                                   |
| Missing values        | Mechanism check (MCAR / MAR / MNAR), then median, mode, KNN, MICE, or a was-missing flag. <i>Fitted fill values are persisted for transform-only export.</i>                              |
| Outliers              | IQR (Tukey) + modified z-score (Hampel / MAD); Isolation Forest for multivariate rows.                                                                                                    |
| Scaling               | Normality test (Shapiro-Wilk / KS), then Standard, Robust, MinMax, Box-Cox, Yeo-Johnson, or log1p. <i>Fitted scaler params (mean, std, center, lambda...) are persisted.</i>              |
| Correlation & leakage | Pearson / Spearman redundancy; ID-uniqueness and target-leak detection.                                                                                                                   |
| Encoding              | Cardinality-routed: label, ordinal, one-hot, frequency, or <b>smoothed</b> target encoding. <i>Fitted maps, frozen category sets, and M-estimate smoothed target means are persisted.</i> |
| Drift (monitoring)    | Population Stability Index (PSI) + Kolmogorov-Smirnov two-sample test.                                                                                                                    |

Every recommendation is shown as a decision card with a confidence score and a plain-English reason, so nothing is applied without your understanding and approval.

## 7. Dataset Size Limits

**PreProAuto** sets limits based on the machine's available RAM at request time — not a fixed number. The reason is that pandas loads the entire DataFrame into memory, expanding a raw

CSV file by approximately 2–10× (typically 6× for mixed-type data).

| Limit type                                                       | Formula                                                                       | Typical value<br>(8 GB laptop) | Hard ceiling |
|------------------------------------------------------------------|-------------------------------------------------------------------------------|--------------------------------|--------------|
| Streamable (profiling, scaling, encoding, IQR outliers, filters) | $\min(10 \text{ GB}, \max(200 \text{ MB}, \text{available\_RAM} * 0.55 * 4))$ | ~1.8 GB raw file               | 10 GB        |
| Heavy in-memory (KNN, MICE, Isolation Forest)                    | $\max(100 \text{ MB}, \text{available\_RAM} * 0.55 / 5)$                      | ~180 MB raw file               | 10 GB        |

**Practical guidance:** For datasets under 200 MB, all features including KNN/MICE and Isolation Forest work on any modern laptop. For 200 MB–2 GB, profiling, scaling, encoding, and export work fine; KNN/MICE may be unavailable — use median/mode instead. Above 2 GB, use Dask or Spark for the heavy operations.

---

## SECTION 2

### Complete User Guide — Notebook & Web UI

---

#### 8. Installation

---

Base install (offline — no AI features, but fully functional):

```
pip install prepro-auto
```

With a specific AI provider:

```
pip install prepro-auto[groq]      # Groq -- free tier, fastest
pip install prepro-auto[openai]    # OpenAI / GPT-4
pip install prepro-auto[anthropic] # Anthropic Claude
pip install prepro-auto[gemini]    # Google Gemini
pip install prepro-auto[mistral]   # Mistral
pip install prepro-auto[ai]       # All five providers at once
```

**Important for Jupyter users:** after installing in a notebook cell with `%pip install prepro-auto`, restart the kernel before importing. The `prepro_auto` command in a Jupyter cell prints the module object — it does **NOT** start the server. See the *Web UI — Step-by-Step Guide* section for the correct way to start the web UI; inside a notebook use `prepro_auto.quickclean(...)`, `launch_file(...)` or `launch(df)` instead.

#### 9. Data Privacy

---

##### What leaves your machine?

**The preprocessing engine is 100% local.** Profiling, cleaning, scaling, encoding, drift, export, versioning, and storage all run in-process with no network calls and no telemetry.

The **optional AI features are the only part that uses the network**. When you enable a provider, **PreProAuto** sends **only**: column names and dtypes, a few sample values per column (default 3–4, never the full dataset), and a 2-row before/after preview when the AI assistant applies a transform.

**PII masking (new in b5):** Before any sample value leaves the machine, `app/services/privacy.py` redacts: emails (→ «email»), phone numbers (→ «phone»), long ID-like numbers (→ «id»), card-like patterns (→ «card»). Columns whose names suggest PII (email, ssn, name, address, phone, dob, account, patient, ...) have **all** sample values replaced with «redacted».

**Two config controls** in `app/core/config.py`:



LLM\_MASK\_PII            Default:  
True

Redact PII-looking  
values before egress

LLM\_SEND\_VALUES        Default:  
True

Set False: send no raw  
values at all

**Will the provider “learn” my data?** Major providers generally do not train on API traffic; several offer zero-retention modes. Check your provider’s data-processing terms. **For zero network egress**, leave AI disabled, or point it at a local OpenAI-compatible endpoint (e.g. Ollama).

## 10. Notebook Interface — All Functions

All notebook functions live in the `prepro_auto` module. Import once and use throughout your notebook.

```
prepro_auto.quickclean(data, *, target=None, threshold=0.90, apply_all=False, stages=None, verbose=True)
```

**Returns:** CleanResult (with `.df`, `.report`, `.job_id`)

**The fastest path — one call, no browser.** Runs the entire engine headlessly and applies every decision at or above `threshold` confidence. Pass `apply_all=True` for full auto-pilot. `data` may be a pandas DataFrame *or* a path to a CSV/Excel/JSON/Parquet file.

```
import prepro_auto

result = prepro_auto.quickclean("sales.csv", target="churn")
df = result.df                    # cleaned, model-ready DataFrame
print(result)                    # per-stage report: applied vs. left for review

X = df.drop(columns=["churn"]); y = df["churn"]
model.fit(X, y)

# Full auto-pilot:
result = prepro_auto.quickclean("sales.csv", target="churn", apply_all=True)
```

```
prepro_auto.launch_file(file_path, *, domain='general', port=None, open_browser=False)
```

**Returns:** Session

Recommended starting point. Reads a file from disk with auto-encoding and auto-delimiter detection. Handles UTF-8, Latin-1, cp1252, BOM-prefixed files silently.

```
session = prepro_auto.launch_file(r'C:\data\sales.csv')
# Click: http://127.0.0.1:8721/workbench?job=...
```

```
prepro_auto.launch(df, *, domain='general', port=None, open_browser=False)
```

**Returns:** Session

Pass an existing pandas DataFrame directly — no file I/O. The original `df` is never modified; `session.current()` returns the cleaned version.

```
import pandas as pd, prepro_auto
df = pd.read_csv('data.csv')
session = prepro_auto.launch(df)
cleaned = session.current()
```

```
prepro_auto.Session(job_id, port)
```

**Returns:** Session

Reconnect to an existing session by ID and port.

```
session = prepro_auto.Session('64c15bf6-...', port=8721)
```

```
prepro_auto.set_api_key(provider, api_key, *, model=None)
```

**Returns:** dict with `ok/verified/provider/model/reason`

Configure an AI provider for the current session. Held in memory only — never written to disk.

```
result = prepro_auto.set_api_key('groq', 'gsk...')
# result: {ok: True, verified: True, provider: 'groq', model: 'llama-3.3-70b-
          versatile'}
```

## Session object methods

```
session.current()
```

**Returns:** pandas DataFrame

Returns the current active-version DataFrame. Thread-safe.

```
X = session.current().drop(columns=['target'])
y = session.current()['target']
model.fit(X, y)
```

```
session.update(df)
```

**Returns:** the same Session

Push a notebook-edited DataFrame back to the workbench as a new undoable version.

```
import numpy as np
df2 = session.current()
df2['log_price'] = np.log1p(df2['price'])
session.update(df2)
```

`session.url` / `session.job_id` / `session.port`

```
print(session.url)      # http://127.0.0.1:8721/workbench?job=64c15bf6-...
print(session.job_id)   # UUID for direct REST API calls
print(session.port)     # e.g. 8721
```

## Beginner to Advanced — Notebook Progression

| Level        | What you do                                                                  | Code pattern                                                                                                                                        |
|--------------|------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| Fastest      | One headless call — no browser, no clicking                                  | <code>result = prepro_auto.quickclean(path, target='y')</code><br><code>df = result.df</code>                                                       |
| Beginner     | Load a file, open the workbench, clean visually, get the clean DataFrame     | <code>session = prepro_auto.launch_file(path)</code><br><code>cleaned = session.current()</code>                                                    |
| Intermediate | Enable AI, use <code>set_api_key</code> , push notebook edits back to the UI | <code>prepro_auto.set_api_key('groq', key)</code><br><code>session.update(my_edited_df)</code>                                                      |
| Advanced     | Reconnect to an existing session, make REST API calls directly               | <code>session = prepro_auto.Session(job_id, port)</code><br><code>import requests;</code><br><code>requests.get(f'{session.url}/api/v1/...')</code> |

## 11. Web UI — Step-by-Step Guide

**Starting the web UI:** Open Command Prompt (not Jupyter) and run:

```
prepro_auto          # default: http://127.0.0.1:8000
prepro_auto --port 9000 # custom port
prepro_auto --host 0.0.0.0 # accessible on local network
```

Then open <http://127.0.0.1:8000/workbench> in your browser.

### Step 1 — Upload

- Drag-and-drop a CSV, Parquet, Excel, or JSON file onto the upload zone.
- Encoding and delimiter are auto-detected — no configuration needed.

### Step 2 — Profile

- **Profiling runs automatically** when you open the Profile step — there is no button to click.
- You see: semantic type, missing rate, unique count, sample values, quality score (0–100).
- No changes are made at this stage — it is observation only.

### Step 3 — View data

- Toggle between **Original (raw)** and **Current** (after any cleaning) views.
- Use **Refresh** to pull the latest state after making changes in other steps.

## Step 4 — Clean (HITL)

- Five stage tabs: **Missing Values** | **Outliers** | **Scaling** | **Correlation** | **Encoding**.
- **Selecting a stage tab runs it automatically** — there is no ‘Run stage’ button. Opening the Clean step auto-runs **Missing Values** first.
- **Quick clean** (one button) runs *all five* stages and applies every recommended decision in a single click — fully undoable. It is the visual equivalent of `prepro_auto.quickclean(..., apply_all=True)`.
- For each card: **Approve**, **Override**, **Skip**, or **Drop column** (or **Approve all pending**).
- Click **‘Execute stage’** to commit — this **auto-advances to and runs the next stage**, refreshes the issue-count badges, and **persists fitted state** for the reproducible export.
- **Undo / Redo** buttons are always available in the sidebar.

## Step 5 — Transform

- 18 preset operations: rename, drop, cast, fillna, filter rows, merge columns, math (+/-/\*/), map values, string ops, regex extract, **group-aggregate**, sort, deduplicate, extract number.
- Expression editor: restricted pandas expressions evaluated in a denylist sandbox.
- AI assist (requires API key + PII masking active): describe in English, AI proposes, you confirm.

## Step 6 — Export

- Download cleaned data as CSV or Parquet.
- Download audit PDF: every decision documented with parameters and before/after stats.
- Download `pipeline.py`: **transform-only**, format-aware (CSV/Parquet/Excel/JSON), no **PreProAuto** dependency. See reproducibility guarantee above.

## 12. All Outputs Explained

| Output                        | How to get it                                              | What's in it                                                                                                  | Where to use it                                                                                  |
|-------------------------------|------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| CleanResult (headless)        | <code>prepro_auto.quickclean (data)</code>                 | <code>.df</code> , <code>.report</code> , <code>.job_id</code>                                                | One-call cleaning; <code>result.df</code> feeds directly into <code>model.fit(X, y)</code> .     |
| Cleaned DataFrame             | <code>session.current()</code> in notebook                 | Active-version pandas DataFrame                                                                               | Feed directly into <code>model.fit(X, y)</code> without saving intermediate files.               |
| Cleaned CSV                   | Export step or GET <code>.../export/data?format=csv</code> | UTF-8 CSV                                                                                                     | Share with teammates, BI tools (Tableau, Power BI), and data repositories.                       |
| Cleaned Parquet               | Export step or GET <code>...?format=parquet</code>         | Column-oriented binary                                                                                        | Large datasets, feature stores, and distributed Spark/Dask pipelines.                            |
| Audit PDF                     | Export step or GET <code>.../export/audit</code>           | Every transformation, parameters, before/after stats, approver, quality delta                                 | Compliance documentation, model cards, audits, and governance handoffs.                          |
| Pipeline script (pipeline.py) | Export step or GET <code>.../export/pipeline</code>        | <b>Transform-only</b> script using stored fitted params. Format-aware input. No <b>PreProAuto</b> dependency. | Production pipelines, Airflow/Prefect workflows, CI/CD automation, and batch preprocessing jobs. |
| Drift report                  | POST <code>/api/v1/drift/compare</code>                    | Per-column PSI + KS verdict with severity bands                                                               | Model monitoring and automated alerts when <code>significant_drift</code> is detected.           |

## 13. AI Providers Setup

AI features are entirely optional. The full preprocessing engine runs offline without any API key. AI adds: plain-English transform proposals, column advice, and multi-turn conversation about your data. All AI egress is PII-masked (see Section 9).

| Provider  | Best for                     | Install                                         | Get key                                                           | Free? |
|-----------|------------------------------|-------------------------------------------------|-------------------------------------------------------------------|-------|
| Groq      | Fast, cheap, testing         | <code>pip install prepro-auto[groq]</code>      | <a href="https://console.groq.com">console.groq.com</a>           | Yes   |
| OpenAI    | GPT-4o, strongest quality    | <code>pip install prepro-auto[openai]</code>    | <a href="https://platform.openai.com">platform.openai.com</a>     | No    |
| Anthropic | Claude — reasoning           | <code>pip install prepro-auto[anthropic]</code> | <a href="https://console.anthropic.com">console.anthropic.com</a> | No    |
| Gemini    | Google integration           | <code>pip install prepro-auto[gemini]</code>    | <a href="https://aistudio.google.com">aistudio.google.com</a>     | Yes   |
| Mistral   | European / OpenAI-compatible | <code>pip install prepro-auto[mistral]</code>   | <a href="https://console.mistral.ai">console.mistral.ai</a>       | No    |

### Three ways to configure a key:

```
# 1. Notebook (safest -- stays in memory only, never written to disk)
prepro_auto.set_api_key('openai', 'sk-...')

# 2. Web UI -- click 'Configure API key' in the sidebar, paste key, click Test &
    apply

# 3. .env file (survives restarts -- never use on shared or hosted machines)
LLM_PROVIDER=groq
GROQ_API_KEY=gsk...
```

## 14. Troubleshooting

**Problem:** `prepro_auto` typed in Jupyter prints the module object

**Solution:** Use `prepro_auto.launch_file(path)` or `prepro_auto.launch(df)` in a notebook. The CLI command only works in a terminal.

**Problem:** `ERR_CONNECTION_REFUSED` at <http://127.0.0.1:8000>

**Solution:** The server is not running. Open Command Prompt and run `prepro_auto`. Wait for the 'Starting PrePro Auto workbench at...' message before opening the URL.

**Problem:** `AttributeError: module 'prepro_auto' has no attribute 'launch_file'`

**Solution:** Old version installed. Run: `pip install --upgrade prepro-auto`

**Problem:** `UnicodeDecodeError` on CSV load

**Solution:** Use `prepro_auto.launch_file(path)` instead of `pd.read_csv(path)`. `launch_file` auto-detects encoding (UTF-8, Latin-1, cp1252, BOM) silently.

**Problem:** KNN or MICE imputation fails on large dataset

**Solution:** Check `GET /api/v1/system/limits` to see `heavy_max_mb` for your machine. Use median/mode imputation instead, or run on a machine with more available RAM.

**Problem:** Pipeline export uses median instead of re-running KNN/MICE on new data

**Solution:** This is intentional and correct. KNN/MICE cannot be deterministically re-applied to arbitrary new rows without re-fitting on (and therefore leaking) the new data. The exported pipeline reuses the **median learned at clean time**. This preserves the train/serve contract. Pickle-sidecar export (which would allow the full imputer to be reused) is on the roadmap.

**Problem:** Drift detection returns ‘no common columns’

**Solution:** Ensure both reference and new datasets have the same column names.

---

**Questions, bugs, or feature requests?**

Open an issue at [https://github.com/Chilliflex/prepro\\_auto](https://github.com/Chilliflex/prepro_auto)

PyPI page: <https://pypi.org/project/prepro-auto>

Security disclosures: see SECURITY.md in the repository.